

Weekly Progress Report

Project: Custom 2D Platformer - Group 3

Date / Week: 30/6 to 4/7 | Week 04

1. Weekly Overview

- **Primary Goal:**
 - Allocate tasks among team members, finalize the team leader role, define individual responsibilities, establish the workflow, and outline the projected project timeline.
 - Initialize the GitHub repository to serve as the centralized collaborative workspace for the team.
 - Reach a consensus on the game's visual assets, core mechanics, overall architecture, and future development direction.
 - Establish the core skeleton and initialization for the project, including the C++/SFML project setup, basic physics (gravity/collision), rendering a static map from a text file, implementing basic keyboard inputs for Mario, and structuring the base Enemy class.
- **Completion Rate:** 80% of the weekly plan completed.

2. Individual Task Breakdown

25125057 - Trần Minh Khoa is responsible for the **Core Engine & Physics**.

- **Completed Tasks:**
 - Added a singleton Game class in `include/Core/Game.h` to ensure only one instance of the game can exist at a time, encapsulating the main window, game loop, and event handling logic.
 - Implemented the Game class methods in `src/Core/Game.cpp`, including the game loop (run), event processing, updating, and rendering, with a fixed timestep and window management using SFML.
 - Updated `src/main.cpp` to start the game by invoking `Game::getInstance().run()`.
 - Modified `CMakeLists.txt` to ensure SFML is built as a static library by setting `BUILD_SHARED_LIBS` to `OFF`.
- **Proof:** <https://github.com/trannhukhai98438/CS202-Group03-2DGame/pull/1>

25125045 - Trần Như Khải is in charge of the **Hero & Items Logic**.

- **Completed Tasks:**
 - Allocated primary tasks among team members.
 - Initialized the shared GitHub repository for the team.
 - Coded the base `Character` class (serving as the parent class for both Hero and Enemy entities).
 - Set up the fundamental structure for the `Mario` class and implemented the `HeroFactory` utilizing the Factory design pattern.
- **Proof:** <https://github.com/trannhukhai98438/CS202-Group03-2DGame/tree/feature/Hero>

25125024 - Đỗ Viết Hoàng Long is handling the **Enemies & AI Design**.

- **Completed Tasks:** Designed the abstract base class `Enemy` using the **Template Method Pattern** to enforce a structured 3-step patrol AI (`checkObstacles`, `move`, `applyAnimation`). Created concrete header structures for `Goomba` and `Koopa`.
- **Note:** The current implementation relies on the base `Character` class. Since the official `Character.h` hasn't been pushed to the main repository yet, compilation might temporarily fail until the `Character` component is integrated.
- **Proof:** <https://github.com/trannhukhai98438/CS202-Group03-2DGame/tree/feature/enemies-ai>

25125056 - Trần Đăng Khoa is assigned to the development of the **Tilemap, HUD & Audio**.

- **Completed Tasks:** no tasks completed yet.

3. Challenges & Solutions

This week had no challenges or conflicts yet.

4. AI Usage Notes:

- Gemini. Gemini 3.1 Pro, Google, gemini.google.com, accessed 22:15 on July 3, 2026, prompt: "Provide a structural guide for building a core game engine in C++, starting with the initialization of the SFML RenderWindow and the implementation of a fixed-timestep game loop.", used for understanding the idea for the core engine for games; AI introduced the frames to capture the states, student researches on the internet for the method, codes and runs.
- Gemini. Gemini 3.1 Pro, Google, gemini.google.com, accessed 22:25 on July 3, 2026, prompt: "Resolve a runtime system error (sfml-system-d-2.dll was not found).", used for debugging; AI suggested to put the DLLs next to the executable, then add `set(BUILD_SHARED_LIBS OFF)` into `CMakeLists.txt` to prevent the error for other member; student applies the method and checks the status.
- Gemini. Gemini 3.1 Pro, Google, gemini.google.com, accessed 22:30 on July 3, 2026, prompt: "Explain the mechanics of the Singleton design pattern and demonstrate how to securely implement it to manage the core Game class without memory leaks.", used for understanding Singleton pattern and applying this pattern into the `Game` class; AI introduced the concept and showed a demonstration for such a pattern, student researches on the internet for the method, codes and runs.
- Gemini. Gemini 3.1 Pro, Google, gemini.google.com, accessed 18:30 on June 20, 2026, prompt: "Divide the game development project tasks among 4 people. Distribute the work evenly (from UI to Core - ensuring all members get an equal share). Create a complete plan for the phases. Note: Within the OOP framework, use SFML 2.x for the render window", used for planning the project structure, task delegation, and outlining the development phases based on OOP principles; AI provided a comprehensive Work Breakdown Structure (WBS) and a 9-week timeline, student used the plan to assign roles to team members and create the weekly report document.
- Gemini. Gemini 3.1, Google, gemini.google.com, accessed 07:30 on June 21, 2026, prompt: "Guide me on creating a GitHub repo for the project: how to create the repo, invite collaborators, create branches (expected to include a dev branch and a docs branch), and basic files for the project (such as readme, weekly report template - latex, etc.)", used for establishing the version control workflow and repository structure for the team; AI outlined step-by-step instructions for repository creation and branching strategies, student initialized the GitHub repository and invited collaborators.

- Gemini. Gemini, Google, gemini.google.com, accessed 18:39 on July 4, 2026, prompt: "Provide a structural guide for building a base Character class using the Factory Method pattern in C++, including concrete Hero products (Mario1, Mario2) with different abilities, and a HeroFactory to instantiate them.", used for establishing the OOP architecture and entity spawning mechanics; AI generated the abstract class and factory templates, student adapted the polymorphism structure for the SFML project, debugged access modifiers, and integrated the code.
- Gemini, Gemini 1.5 Pro, Google, gemini.google.com, accessed 11:15 on July 4, 2026, prompt: "Should we define the Direction struct like this or put it in a class?"; used for refactoring the architecture of the `MoveDirection` structure from a standard struct to an `enum class` for better type safety, and designing a clean header/source directory layout for the entities component; AI provided the foundational OOP concepts and class declarations, student applied these suggestions to code the header files for the Enemy, Goomba, and Koopa classes on the development branch.

5. Next Week's Action Plan

- **Core Mechanics:**
 - Implement basic gravity/collision and integrate complex collision detection (e.g., Mario stomping on Goombas).
 - Finish basic keyboard inputs for movement and jumping, and code Mario's size-changing logic using the Strategy Pattern.
- **Gameplay / Graphics:**
 - Develop the tilemap renderer to display a static Mario level (bricks, pipes) from a text file.
 - Spawn classic items (Mushroom, Coin).
 - Spawn Goombas/Koopas and implement basic walking/bouncing behavior.
- **Expected Deadline:** 11/7/2026